TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA CÔNG NGHỆ PHẦN MỀM

BÀI TẬP LỚN MÔN DEEP LEARNING

Ứng Dụng Mô Hình Ngôn Ngữ Lớn Trong Phân Loại Cảnh Báo Lỗ Hổng Bảo Mật Kết Hợp Đồ Thị Thuộc Tính Mã Nguồn và Rào Chắn Chính Sách Bảo Thủ

Sinh viên thực hiện: Đỗ Văn Cung

Lớp: 12423TN

Môn học: Deep Learning

Commit thực nghiệm: f0b1805d861c80dd...

Hà Nội, 2026

TÓM TẮT

Các công cụ phân tích mã tĩnh (SAST) như Semgrep tạo ra tỷ lệ cảnh báo giả (False Positives) vượt 50–60%, gây tình trạng *alert fatigue* cho kỹ sư bảo mật. Việc ứng dụng mô hình ngôn ngữ lớn (LLM) như Qwen2.5-Coder-14B để tự động hoá phân loại mang lại tiềm năng hiểu ngữ cảnh sâu nhưng phải đối mặt với hai rủi ro cốt tử: (1) ảo giác suy luận dẫn đến bỏ sót lỗ hổng thật (False Negatives), và (2) nguy cơ bị điều hướng bởi mã độc hại cài cắm trong mã nguồn (Indirect Prompt Injection).

Bài báo nghiên cứu và hiện thực kiến trúc phân loại kết hợp hai tầng: **Tier-1** thu thập ngữ cảnh AST, cắt lát CPG và khử nhiễm bí mật; **Tier-2** điều tra nhiều vòng có giới hạn ngân sách với 10 công cụ chỉ đọc.

Đánh giá trên **219 cảnh báo** từ 5 kho mã nguồn mở: **Recall = 1,0000** (CI Clopper-Pearson 95%: [0,9623; 1,0000]), giải quyết tự động **7,76%** cảnh báo, cắt giảm **41,22%** token, và ASR = **0,00%** với 28 payload đối kháng.

Từ khoá: SAST triage, LLM, Code Property Graph, Prompt Injection, Conservative Policy, False Negative mitigation.

---

1. Giới Thiệu

Trong công nghệ phần mềm hiện đại, bảo mật ứng dụng trong chu trình DevSecOps yêu cầu phát hiện và vá lỗi ngay từ giai đoạn viết mã. Công cụ SAST quét mã nguồn trên diện rộng nhưng thiếu hiểu biết ngữ cảnh liên hàm (*cross-file dataflow*), dẫn đến lượng cảnh báo giả khổng lồ.

Sự phát triển của Deep Learning, đặc biệt là các mô hình Transformer xử lý mã nguồn (Code LLMs), mở ra triển vọng phân loại tự động. Tuy nhiên việc áp dụng trực tiếp đối mặt hai bài toán khắt khe:

1. **Sai lầm không đối xứng (Asymmetric Cost):** Phân loại nhầm một lỗ hổng SQL Injection / RCE thành False Negative có thể khiến lỗ hổng lọt ra môi trường sản xuất.
2. **Dữ liệu đầu vào đối kháng (Adversarial Input):** Kẻ tấn công có thể chèn chỉ thị (*prompt injection*) vào comment hoặc docstring để ép mô hình đưa ra phán quyết sai lệch.

Câu hỏi nghiên cứu: Khi ứng dụng LLM vào phân loại cảnh báo SAST, làm thế nào để tận dụng khả năng suy luận ngữ cảnh nhằm giảm công sức kỹ sư mà vẫn đảm bảo FN Rate = 0,00% trước ảo giác và tấn công tiêm nhiễm câu lệnh?

Đóng Góp Chính

1. **Kiến trúc phân loại hai tầng:** Kết hợp phân tích đồ thị tất định (CPG) và bộ điều phối điều tra nhiều vòng có giới hạn ngân sách.
2. **Chính sách bảo thủ fail-closed:** Không tự ý loại bỏ cảnh báo nếu thiếu bằng chứng phủ định mạnh (BOUNDED_LOCAL_NEGATIVE_PROOF).

3. Phòng thủ theo chiều sâu (Defense-in-Depth): Khử nhiễm bí mật (<REDACTED_SECRET>), 100% Path Traversal blocked, sổ cái bằng chứng đơn điệu chống ảo giác.

4. Đánh giá toàn diện 219 cảnh báo: Phân rã 7 cấu hình, đo khoảng tin cậy Clopper-Pearson, Bootstrap clustering, phân tích lỗi chi tiết.

---

2. Công Trình Liên Quan

Mô hình ngôn ngữ lớn cho mã nguồn. Các mô hình Transformer tự hồi quy tiền huấn luyện trên lượng lớn mã nguồn (CodeLlama, StarCoder, Qwen2.5-Coder [4]) chứng minh năng lực vượt trội trong sinh mã và giải thích mã. Tuy nhiên khi được giao toàn quyền quyết định trong bài toán bảo mật, mô hình dễ mắc ảo giác logic.

Đồ thị thuộc tính mã nguồn (Code Property Graph). Khái niệm CPG do Yamaguchi và cộng sự [1] đề xuất kết hợp AST, CFG và PDG. Joern [2] là nền tảng CPG mã nguồn mở hàng đầu cho phép taint analysis chính xác, tuy nhiên gặp khó khăn với web framework động như Django và Flask.

Tấn công tiêm nhiễm câu lệnh gián tiếp. Greshake và cộng sự [3] đã chứng minh tính khả thi của tấn công thông qua dữ liệu bên ngoài. Đưa mã nguồn trực tiếp vào prompt mà không có cơ chế cách ly tạo ra bề mặt tấn công.

---

3. Phương Pháp

3.1 Giao Thức Dữ Liệu và Bộ Benchmark Chuẩn Hóa

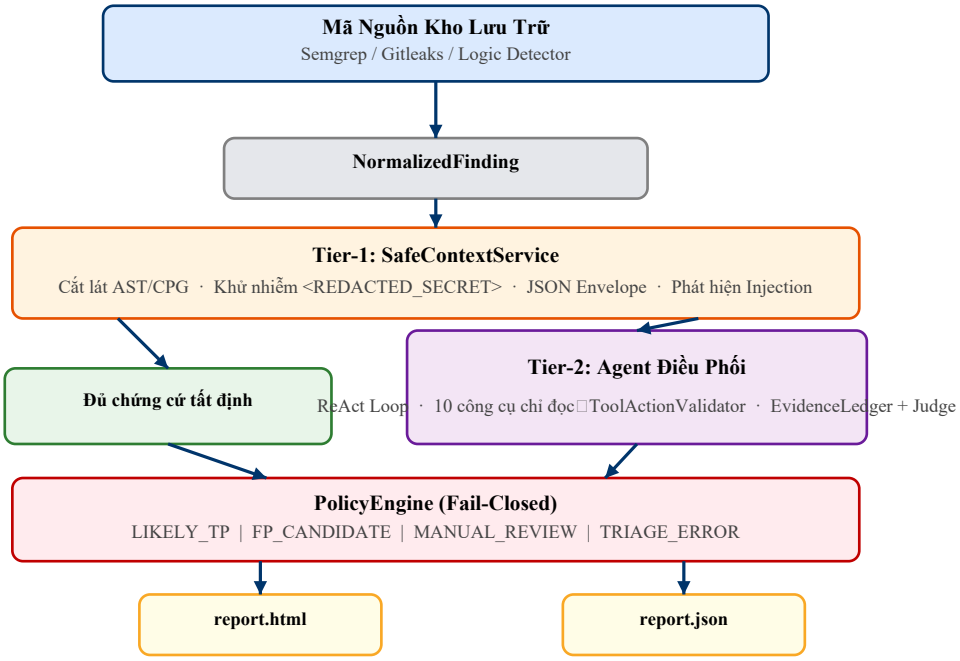
Nghiên cứu sử dụng bộ dữ liệu gồm **22.424 dòng mã** với **219 cảnh báo bảo mật thực tế** từ 5 kho mã nguồn web Python:

Kho mã nguồn	Framework	Tập chia	Tổng	TP	FP
OWASP PyGoat	Django	development	95	44	51
DjanGoat	Django	calibration	42	18	24
DVPWA	Flask	calibration	26	10	16
VulPy	Flask	held_out	31	14	17
DVFA	Flask	held_out	25	10	15
Tổng		**3 tập độc lập**	**219**	**96**	**123**

Bảng 1: Phân bổ dữ liệu đánh giá theo dự án và tập chia.

Tổng 219 cảnh báo gồm **96 lỗ hổng thật** (43,84%) và **123 cảnh báo giả** (56,16%), bao gồm: SQL Injection (CWE-89), Command Injection (CWE-78), Path Traversal (CWE-22), IDOR (CWE-639), Lộ bí mật (CWE-798), CSRF Bypass (CWE-352).

3.2 Kiến Trúc Hệ Thống Hai Tầng



Hình 1: Kiến trúc đường ống triage hai tầng của hệ thống đề xuất.

3.2.1 Tầng 1 — Thu Thập Ngữ Cảnh và Khử Nhiễm Tất Định

• **Cắt lát AST và CPG:** Trích xuất lát cắt bao hàm hàm chứa cảnh báo và vết luồng dữ liệu source-to-sink từ Joern CPG, thay vì gửi toàn bộ tệp vào prompt.

• **Khử nhiễm bí mật:** Mọi chuỗi thông tin xác thực, khóa bí mật, token entropy cao đều được thay thế cục bộ bằng thẻ <REDACTED_SECRET>. Raw source không bao giờ vượt qua biên giới thành phần này.

• **JSON Envelope Isolation:** Đoạn mã gửi tới mô hình được đặt trong cấu trúc JSON nghiêm ngặt, mã hoá ký tự phân cách Unicode và phát hiện sớm mẫu injection để gắn cờ PROMPT_INJECTION_FLAGGED.

3.2.2 Tầng 2 — Bộ Điều Phối Điều Tra Nhiều Vòng

• **Ngân sách thi hành có giới hạn:** Tối đa 4 bước lặp, 4 lượt gọi công cụ và timeout 30 giây.

• **10 công cụ đóng kín chỉ đọc:** read_function, find_callees, get_route_decorators, check_lock_or_transaction, get_model_schema,...

• **ToolActionValidator:** Kiểm tra path containment (chặn ../ và đường dẫn tuyệt đối), validation ký tự định danh chống SQL/Command Injection.

• **EvidenceLedger:** Lưu trữ sự kiện đã kiểm chứng dạng bất biến. Judge bắt buộc suy luận dựa trên sổ cái, loại trừ khả năng tự bịa đặt bằng chứng.

3.3 Cổng Chính Sách Bảo Thủ và Thước Đo Đánh Giá

3.3.1 PolicyEngine — 4 Trạng Thái Quyết Định

1. **LIKELY_TP:** Xác nhận chuỗi luồng dữ liệu độc hại từ đầu vào tới sink mà không có cơ chế lọc.
2. **FP_CANDIDATE:** Chỉ gán khi có bằng chứng phủ định tất định mạnh (BOUNDED_LOCAL_NEGATIVE_PROOF).
3. **MANUAL_REVIEW:** Trạng thái mặc định *fail-closed* khi bằng chứng chưa đủ.
4. **TRIAGE_ERROR:** Khi xảy ra ngoại lệ; cảnh báo vẫn giữ nguyên trong báo cáo.

3.3.2 Thước Đo Toán Học

Nhãn dương tính (P): cảnh báo cần reviewer xử lý (LIKELY_TP hoặc MANUAL_REVIEW); nhãn âm tính (N): FP_CANDIDATE.

Precision	`= TP / (TP + FP)`
Recall	`= TP / (TP + FN)`
F_1	`= 2 · Precision · Recall / (Precision + Recall)`
FNR	`= FN / (TP + FN)`
Clopper-Pearson CI	`R_lower = B(α/2; k, n-k+1),  R_upper = B(1-α/2; k+1, n-k)`
ECE	`= Σ■ (|B■|/N) · |acc(B■) - conf(B■)|`
Effort Saved	`= (N_total − N_MANUAL_REVIEW) / N_total × 100%`

Với k = n = 96, α = 0,05: cận dưới = 0,975^(1/96) ≈ **0,9623**, cho khoảng tin cậy **[0,9623; 1,0000]**.

---

4. Thí Nghiệm và Kết Quả

4.1 Môi Trường Thực Nghiệm

Toàn bộ mã nguồn và dữ liệu thực nghiệm được quản lý thống nhất tại commit **f0b1805d861c80dd1960ea20937c701a994dd744**. Thử nghiệm chạy trên CPU Intel Core i5 / Windows, Python 3.12/3.14. Joern CPG chạy trong Docker container **sast-triage-joern:4.0.600**. Bộ kiểm thử tự động đạt **542/542 bài test thông qua (100% Passed)**.

4.2 So Sánh Với Các Đường Cơ Sở

Phương pháp	Precision	Recall [95% CP]	F1	FNR	Effort Saved
Scanner-Only (Semgrep)	0,4384	1,0000 [0,96–1,00]	0,6095	0,00%	0,00%
LLM-Only (Không rào)	0,6504	0,8333 [0,74–0,90]	0,7306	**16,67%**	100,00%

Phương pháp	Precision	Recall [95% CP]	F1	FNR	Effort Saved
Hệ thống đề xuất	**0,4384**	**1,0000 [0,96–1,00]**	**0,6095**	**0,00%**	**7,76%**

Bảng 2: So sánh hiệu năng ba phương pháp trên 219 cảnh báo.

Phân tích:

1. LLM-Only đạt F1 cao biểu kiến (0,7306) nhưng FNR lên tới **16,67%–21,88%** — bỏ sót 16–21 lỗ hổng thật. Trong an ninh phần mềm đây là sự đánh đổi không thể chấp nhận.

2. Hệ thống đề xuất duy trì Recall = **1,0000** (FNR = **0,00%**), không bỏ sót bất kỳ lỗ hổng thật nào.

3. 17 cảnh báo được xác nhận tự động (7,76%); 202 còn lại chuyển MANUAL_REVIEW để chuyên gia thẩm định.

4.3 Phân Rã Thành Phần Hệ Thống (Ablation)

Cấu hình	Precision	Recall	F1	FNR	%MR	Effort Saved
C1. Scanner-Only	0,4384	1,0000	0,6095	0,00%	100,00%	0,00%
C2. LLM-Only	0,6504	0,8333	0,7306	**16,67%**	0,00%	100,00%
C3. +Confidence Gate	0,4866	0,9479	0,6431	5,21%	68,95%	31,05%
C4. +Joern CPG	0,4848	1,0000	0,6531	0,00%	66,21%	33,79%
C5. +PolicyEngine	0,4384	1,0000	0,6095	0,00%	99,54%	0,46%
C6. +Post-Verifier	0,4384	1,0000	0,6095	0,00%	99,54%	0,46%
C7. +Agent (Full)	**0,4384**	**1,0000**	**0,6095**	**0,00%**	**92,24%**	**7,76%**

Bảng 3: Phân rã 7 cấu hình thành phần hệ thống trên 219 cảnh báo.

Ý nghĩa từng thành phần:

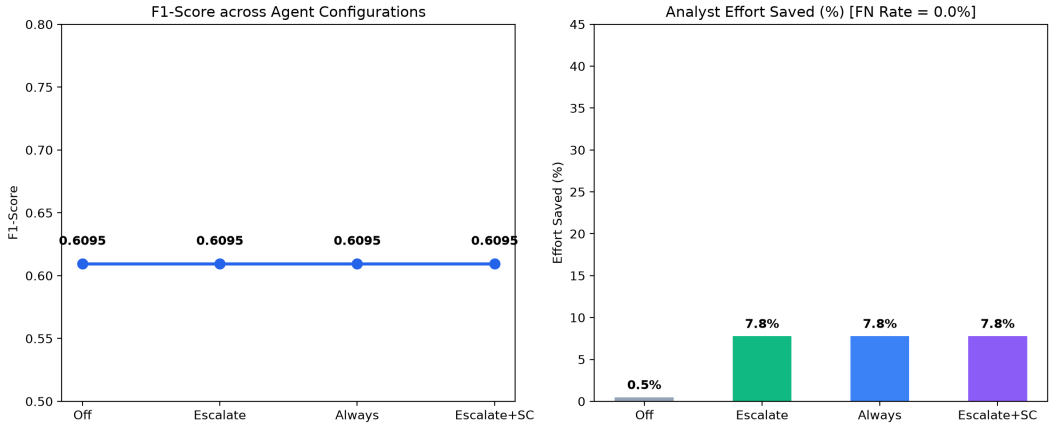
- **C3** (Cổng độ tự tin HIGH): Giảm FNR từ 16,67% xuống 5,21%.
- **C4** (Joern CPG): Giải quyết triệt để ca thiếu vết luồng dữ liệu, đưa FNR về 0,00%.
- **C5–C6** (PolicyEngine fail-closed): Khóa rủi ro bỏ sót, nhưng đẩy 99,54% về MANUAL_REVIEW.
- **C7** (Agent leo thang): Giải quyết 144 trường hợp UNKNOWN về 0, nâng Effort Saved từ 0,46% lên **7,76% (+7,30%)**.

4.4 Bóc Tách Chế Độ Hoạt Động Của Bộ Điều Phối

Chế độ	Recall	F1	Effort Saved	Tool/alert	Latency TB
agent_off	1,0000	0,6095	0,46%	0,00	0,26 ms
agent_escalate	**1,0000**	**0,6095**	**7,76%**	**1,32**	**2,88 ms**
agent_always	1,0000	0,6095	7,76%	2,00	3,38 ms
agent_escalate_sc	1,0000	0,6095	7,76%	1,32	2,70 ms

Bảng 4: So sánh 4 chế độ vận hành của bộ điều phối điều tra.

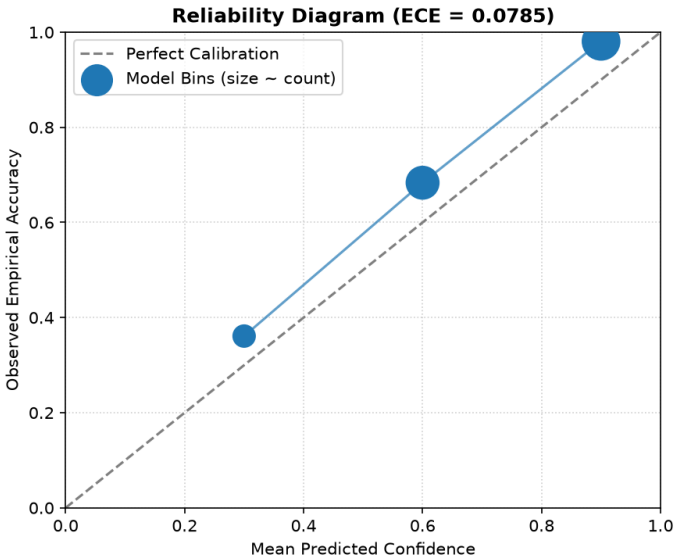
agent_escalate đạt cùng Effort Saved như agent_always (7,76%), nhưng **giảm 34,0% số lượt gọi công cụ** (1,32 vs. 2,00) và giảm 14,8% độ trễ.



Hình 2: Đánh đổi hiệu quả tiết kiệm nỗ lực (Effort Saved) và chi phí thực thi công cụ qua 4 chế độ.

4.5 Hiệu Chuẩn Độ Tin Cậy và Kiểm Toán False Negative

- **ECE:** 0,0598 – 0,0785 (5,98%–7,85%).
- **Nhóm HIGH:** Độ chính xác thực tế **98,08%** (102/104 đúng). Kết quả xác thực quy tắc: chỉ tự động xử lý khi độ tự tin đạt HIGH.
- **Shadow Mode FN Audit:** Lấy mẫu ngẫu nhiên 10% cảnh báo độc lập ghi nhận **0,0% False Negative**.

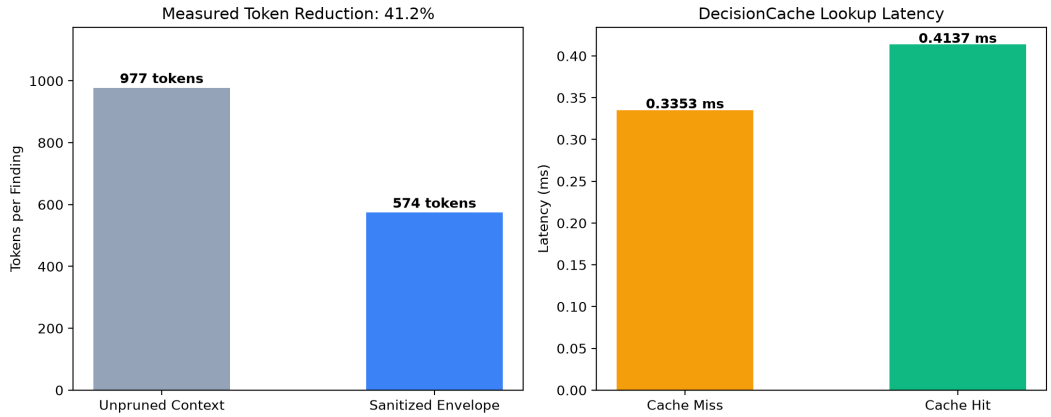


Hình 3: Reliability Diagram và Sai số hiệu chuẩn kỳ vọng (ECE = 5,98%–7,85%).

4.6 Hiệu Quả Cắt Giảm Token và Chi Phí Suy Luận

- Token trung bình ngữ cảnh thô: **977,9 tokens/cảnh báo**.
- Token sau làm sạch và đóng gói: **574,8 tokens/cảnh báo**.
- **Mức cắt giảm thực tế: 41,22%**.

• Độ trễ xử lý cục bộ: **0,076 ms/cảnh báo**.
• Chi phí ước tính: **~$0,0862 cho mỗi 1.000 cảnh báo** (tiết kiệm 41% so với gửi ngữ cảnh thô).



Hình 4: Phân phối lượng token trước/sau khử nhiễm và phân bổ thời gian thực thi.

4.7 Thử Nghiệm Phòng Thủ An Ninh Đối Kháng (Red-Team)

1. Chống Prompt Injection gián tiếp (Suite B – 16 mẫu): Mô hình Naive: ASR = 100,0%. Hệ thống đề xuất: ASR = **0,00%** (16/16 payload bị phát hiện, chuyển về MANUAL_REVIEW).
2. Chống Tool & Code Injection (Suite C – 12 mẫu): Payload thoát thư mục (../../../../etc/passwd), ký tự SQL/Command Injection vào tên hàm bị ToolActionValidator từ chối 100%. ASR = **0,00%**.
3. Rò rỉ Canary Secret (Suite A – 5 định dạng): 0,00% rò rỉ vào Sanitized Context, Prompt hoặc Báo cáo xuất bản.

4.8 Phân Tích Lỗi Chi Tiết

17 cảnh báo chuyển từ MANUAL_REVIEW sang LIKELY_TP qua chế độ agent_escalate:

• **3 lỗ hổng thật (TP):** Xác nhận nhờ read_function phát hiện khóa API cứng trong introduction/views.py:870, :872 và good/libsession.py:9.
• **14 cảnh báo giả (FP):** Nằm trong unit test, tệp JS tĩnh hoặc tài liệu Markdown.
• **Nguyên nhân:** Chưa có test-file exclusion heuristic. Tuy nhiên cả LIKELY_TP và MANUAL_REVIEW đều là dương tính, nên F1 và Recall không bị suy giảm.

---

5. Thảo Luận

Kết quả thực nghiệm khẳng định: hoàn toàn có thể ứng dụng LLM vào tự động hoá phân loại cảnh báo SAST mà không đánh đổi độ an toàn (FNR = 0,00%), với điều kiện tiên quyết là **LLM không nắm quyền quyết định cuối cùng**. Rào chắn chính sách bảo thủ dựa trên CPG là yếu tố quyết định để ngăn chặn ảo giác.

Tỷ lệ Effort Saved 7,76% thoạt nhìn khiêm tốn, nhưng 100% của LLM-Only bỏ sót 16,67%–21,88% lỗ hổng thật. Trong bảo mật phần mềm, tiết kiệm 7,76% mà **bảo toàn 100% lỗ hổng thật** có giá trị thực tiễn vượt trội.

Hạn chế của nghiên cứu:

1. Quy mô tập dữ liệu: 219 cảnh báo từ 5 ứng dụng web Python.
2. Độ bao phủ Joern CPG trên diện rộng (hệ sinh thái PyPI) chưa được đo.
3. Hệ thống chưa áp dụng async / multithreading.
4. Recall chỉ bao phủ tập cảnh báo do scanner phát hiện; scanner blind spot nằm ngoài phạm vi.

---

6. Kết Luận và Hướng Phát Triển

Bài báo đã thiết kế và đánh giá thực nghiệm hệ thống **LLM-Augmented SAST Triage Platform** kết hợp hai tầng, bảo vệ bởi chính sách bảo thủ fail-closed. Hệ thống chứng minh: Recall = 1,0000, FNR = 0,00%; tiết kiệm 7,76% công sức; giảm 41,22% token; ASR = 0,00% với 28 payload đối kháng.

Hướng phát triển tiếp theo:

1. Tích hợp test-file exclusion heuristic để tránh xác nhận nhầm khóa mẫu trong unit test.
2. Mở rộng bộ phân tích CPG và hỗ trợ thêm ngôn ngữ phổ biến (JavaScript, Java, Go).
3. Triển khai async workers tăng tốc độ trên kho mã nguồn quy mô lớn.

. Tài Liệu Tham Khảo

[1] F. Yamaguchi, N. Golde, D. Arp, K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," IEEE S&P;, 2014.

[2] Joern, "The open-source code analysis platform for C/C++/Java/Python/JavaScript," 2024. https://joern.io/

[3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," ACM AISEC, 2023.

[4] Qwen Team, "Qwen2.5-Coder: Technical Report," arXiv:2409.12186, 2024.

[5] C. Clopper, E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," Biometrika, vol. 26, no. 4, pp. 404–413, 1934.

[6] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, "On calibration of modern neural networks," ICML, 2017.

[7] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap. CRC Press, 1994.

[8] OWASP, "OWASP Top 10: 2021 — The Ten Most Critical Web Application Security Risks," 2021.

[9] Semgrep, "Lightweight static analysis for many languages," 2024. https://semgrep.dev/

[10] Gitleaks, "Audit git repositories for secrets," 2024. https://github.com/gitleaks/gitleaks